How the Campaign Page Saves Data to Supabase

Oct 12, 2026 · @Shivtf

Overview

The Campaign page never talks to Supabase directly. It calls the dashboard's own Next.js API routes (`/api/campaigns`), and those routes write to the Supabase table `public.campaign` on the server with the service role key.

Every request goes through four layers:

1. **The page** (`CampaignsView.js`): the form, the cards and the on/off switch.
2. **The client helper** (`core/campaigns.js`): small functions that send the request with the signed-in user's token attached.
3. **The API routes** (`app/api/campaigns` , run on Vercel): check who is calling, validate the data and turn it into a database row.
4. **Supabase** (`campaign` table): saves the row and returns it, and the page updates its card.

The Render backend (`dashboardbackend`) is not involved. Only its folder holds the SQL file that creates the table.

The campaign table

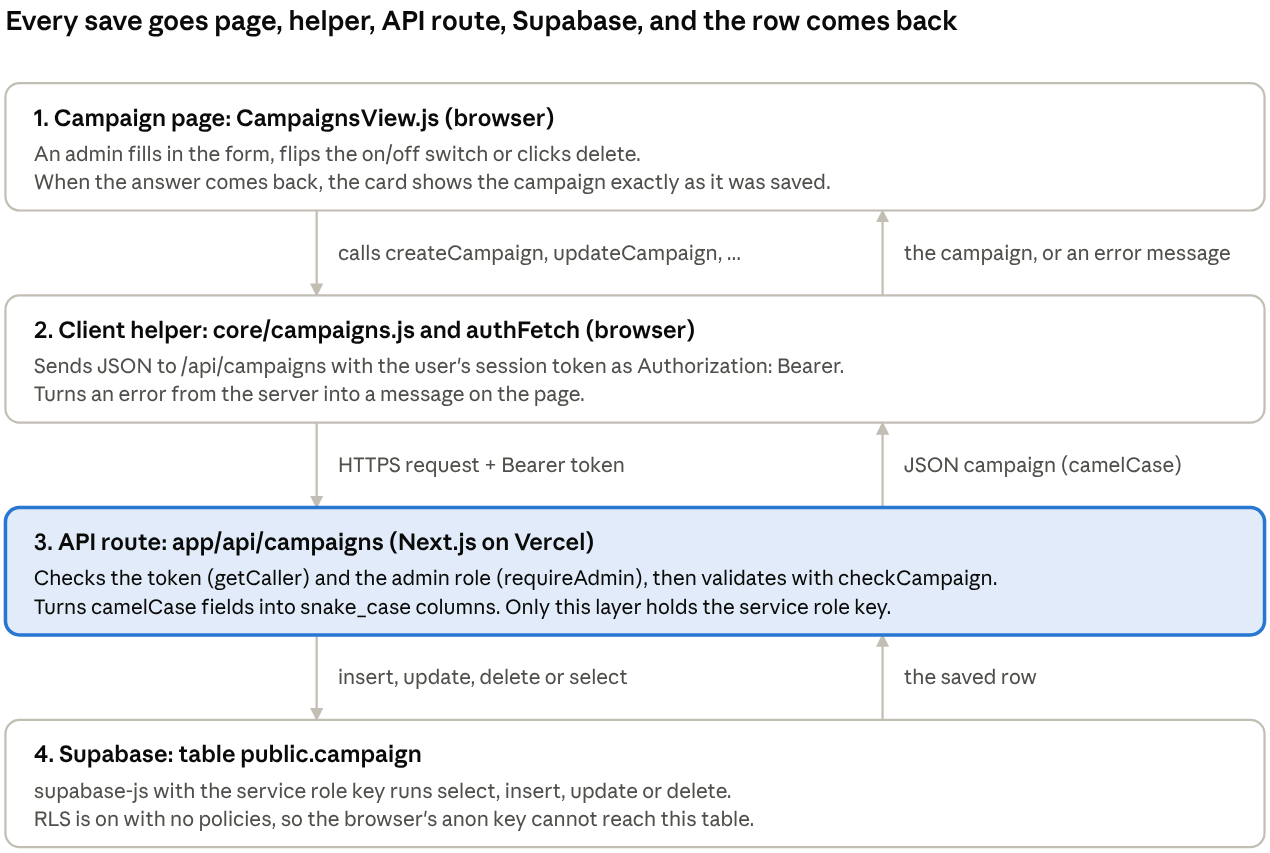
The data lives in `public.campaign` . It is created by `dashboardbackend/migrations/003_campaign.sql` , which you run by hand in the Supabase SQL editor; nothing creates it automatically. The page uses camelCase names and the database uses snake_case, and the server converts between them.

Column	Type	Page field	Rules
`id`	uuid	`id`	Made by the database
`campaign_name`	text	`campaignName`	Required, 1–200 characters

Column	Type	Page field	Rules
`campaign_detail`	text	`campaignDetail`	Optional, up to 5000 characters (checked by the server)
`start_date`	date	`startDate`	Required, `YYYY-MM-DD`
`end_date`	date	`endDate`	Required, on or after `start_date`
`is_active`	boolean	`isActive`	Defaults to `true`
`created_at`	timestamptz	`createdAt`	Set by the database
`updated_at`	timestamptz	`updatedAt`	Set by the server on every edit

Row Level Security is on with no policies, so the browser's anon key cannot read or write the table at all.

Step by step



campaign save path · 4 layers, request down, saved row back up

The request travels down the left arrows and the saved row comes back up the right ones. Supabase sees only the server and its service key, never the browser.

Each operation

The page can do five things, and each one maps to a single request and a single Supabase call.

Action on the page	Helper in `core/campaigns.js`	Request	Supabase call	Who can do it
Load the list	`listCampaigns()`	`GET /api/campaigns`	`select('*')`, newest start date first	Anyone signed in
Add a campaign	`createCampaign(c)`	`POST /api/campaigns`	`insert(row)`	Admins
Edit a campaign	`updateCampaign(id, changes)`	`PATCH /api/campaigns/{id}`	`update(row).eq('id', id)`	Admins
Turn on or off	`updateCampaign(id, { isActive })`	`PATCH /api/campaigns/{id}`	`update({ is_active }).eq('id', id)`	Admins
Delete	`deleteCampaign(id)`	`DELETE /api/campaigns/{id}`	`delete().eq('id', id)`	Admins

An example body for adding a campaign:

```
{
  "campaignName": "Diwali launch",
  "campaignDetail": "Three posts a week on dev.to",
  "startDate": "2026-10-20",
  "endDate": "2026-11-05",
  "isActive": true
}
```

Every write returns the saved row (`.select('*')`), so the card shows exactly what the database stored. An edit sends only the fields that changed, and the server checks only those.

Security

Only the server holds the key that can write to the table, and it checks every caller before using it.

- **Who is calling:** `authFetch` sends the user's Supabase session token as `Authorization: Bearer <token>`. `getCaller` checks that token with Supabase Auth. A missing or invalid token, or a removed user, gets `401 Not signed in.`
- **Admins only for changes:** `requireAdmin` also checks `app_metadata.role === 'admin'`. Anyone else gets `403 Admins only.` Reading the list only needs a signed-in user.
- **Service key stays on the server:** `adminClient()` reads `SUPABASE_SERVICE_ROLE_KEY` from the server environment. The file imports `server-only`, so the build fails if browser code imports it.
- **RLS as a backstop:** the table has RLS on and no policies. Even if someone took the public anon key from the browser, they could not touch campaigns.
- **Input checks:** `checkCampaign` rejects empty names, overlong text, dates that don't exist (such as 2026-02-30) and end dates before start dates. The `{id}` in the URL must be a UUID, or the route answers 404.

Why it works this way

Campaigns go through Next.js routes on Vercel so that a change only needs a Vercel deploy. The Render backend never has to change. The Users page and the weekly blog files already work the same way, so the dashboard has one pattern for its own data.

There is one trade-off. A new column or table means a new SQL file under `dashboardbackend/migrations/`, which someone has to run by hand in the Supabase SQL editor. Until it runs, the routes return a clear error naming the file to run.

Key files

All paths are relative to the repo root.

File	Runs in	What it does
`dashboard/app/(app)/campaign/page.js`	Browser	The `/campaign` route; renders `CampaignsView`
`dashboard/components/CampaignsView.js`	Browser	Cards, filters, search, the add/edit dialog, the on/off switch, delete
`dashboard/core/campaigns.js`	Browser	`listCampaigns`, `createCampaign`, `updateCampaign`, `deleteCampaign`
`dashboard/core/auth.js`	Browser	`authFetch`: adds the Bearer token and turns errors into messages
`dashboard/app/api/campaigns/route.js`	Server (Vercel)	`GET` list, `POST` create
`dashboard/app/api/campaigns/[id]/route.js`	Server (Vercel)	`PATCH` edit or toggle, `DELETE`
`dashboard/core/server/campaigns.js`	Server (Vercel)	`checkCampaign` validation, `toCampaign` row conversion, `dbError` messages
`dashboard/core/server/supabaseAdmin.js`	Server (Vercel)	`adminClient` (service key), `getCaller`, `requireAdmin`, `handle`
`dashboardbackend/migrations/003_campaign.sql`	Supabase SQL editor	Creates the table, its checks, an index on `start_date`, and RLS